

Libra NPU_Support 分支

仿真工具报告

面向 Agentic RL 后训练的资源规划、成本建模与动态重规划

分析对象	NetX-lab/Libra · NPU_Support
基线提交	365f5e42d23b8641ac32e1adaa6feac6b5aa812b
报告日期	2026-09-09
验证方式	源码审查 + CPU 单元测试 + 全流程示例

结论：该分支的仿真工具以解析成本模型为主，兼容 Sailor/Vidur 外部模拟器，并通过 GRP 全局资源规划器将训练、Rollout、设备容量和运行时指标联动起来。CPU 验证链路已通过；真实 NPU 性能结论仍需在 Ascend 设备、CANN、torch-npu、HCCL 与 vLLM-Ascend 环境中实测校准。

目录

- 报告摘要
- 1 项目定位与分析范围
- 2 仿真工具总体架构
- 3 仿真对象、输入与输出
- 4 内置解析成本模型
- 5 Sailor 与 Vidur 外部模拟器适配
- 6 GRP 搜索与动态重规划
- 7 NPU 运行链路仿真关系
- 8 安装与运行方法
- 9 验证结果与复现实验
- 10 输出结果与可观测性
- 11 适用性、限制与风险
- 12 改进与验收建议
- 13 总结

报告摘要

本报告针对 NetX-lab/Libra 的 NPU_Support 分支进行源码、配置、启动脚本、仿真适配器和测试分析。代码基线为提交 365f5e42d23b8641ac32e1adaa6feac6b5aa812b。报告中的仿真工具包括 Libra 内置解析成本模型、Global Resource Planner (GRP)、Sailor 训练仿真器适配器和 Vidur Rollout 仿真器适配器。

该分支的目标不是单独模拟 CANN kernel，而是在真实训练前估计训练与 Rollout 的端到端瓶颈，搜索训练 TP/PP/DP/micro-batch 和异构 Rollout TP 组合，再根据运行历史、队列压力、OOM 预检和重配置成本决定是否切换资源计划。外部 Sailor/Vidur 通过 JSON/CSV 命令契约接入，外部工具不可用时可以回退到解析模型。

本次复现中，从分支干净快照执行源码编译检查，结果为 compileall: PASS；CPU 友好测试结果为 53 passed, 10 skipped；自带 global_resource_planner_full_flow.py 示例输出 FLOW_OK。这些结果验证了仿真和规划软件逻辑，不代表 Ascend NPU 的真实吞吐、HCCL 多机性能或 vLLM-Ascend 性能。

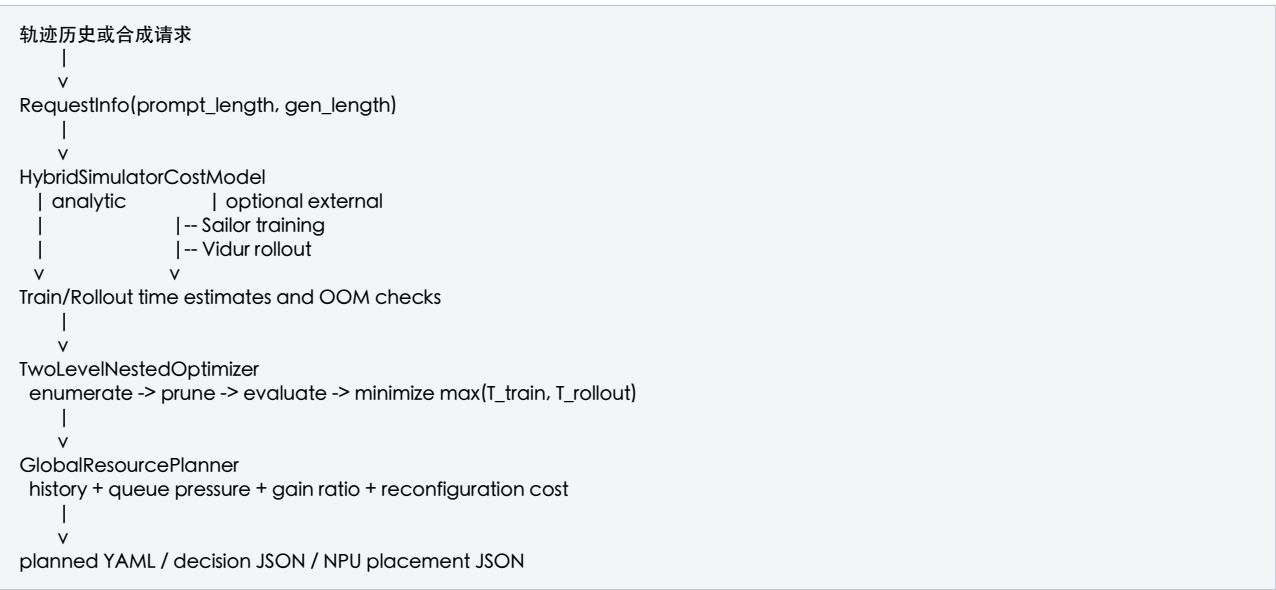
1 项目定位与分析范围

Libra 是面向 Agentic RL 后训练的异步资源管理框架。训练池、Rollout 池和可弹性迁移的 Hybrid 池相互解耦；异步 GRPO 训练器负责采样、优势计算、策略更新和权重同步；C-MLFQ 根据工具调用状态和长度分布将请求路由到不同 TP 的 vLLM 实例；GRP 根据成本模型和运行历史寻找资源划分。

NPU_Support 面向 Ascend NPU，采用 SSH、torchrun 和 HCCL，取代 GPU 分支中的 Slurm/NCCL 运行方式。训练侧提供 Megatron-Core NPU 适配，Rollout 侧提供 vLLM-Ascend 入口，同时保留 disk/restart 兼容路径和官方 HCCL 原位权重刷新路径。

本报告覆盖仿真工具的输入输出、成本模型、搜索算法、外部适配器、动态重规划、NPU 运行关系、安装运行、验证结果和改进建议。本机未执行真实 CANN、torch-npu、HCCL、vLLM-Ascend 或多节点 NPU 任务。

2 仿真工具总体架构



模块	主要职责
infra/cost_model/model.py	训练和 Rollout 解析成本模型、内存估算、OOM 检查
infra/cost_model/optimizer.py	训练并行配置和 Rollout TP 分区搜索
infra/cost_model/simulator_adapters.py	Sailor/Vidur 适配、结果解析和回退
infra/cost_model/global_resource_planner.py	历史、动态触发、候选计划和收益判断
infra/cost_model/preflight_planner.py	启动前计划生成
examples/global_resource_planner_full_flow.py	命令契约端到端示例
scripts/run_global_resource_preflight.py	生成规划 YAML 和决策 JSON
scripts/plan_unrestricted_grp_device_placement.py	将计划装箱到主机和 NPU 设备

3 仿真对象和输入输出

3.1 请求

每条请求被归一化为：

```
RequestInfo(prompt_length=<输入 token 数>, gen_length=<生成 token 数>)
```

总长度为两者之和。请求可以来自真实训练 history、历史 JSONL 或合成输入。对于 Agentic RL，输入长度、生成长度、工具返回状态和长尾请求会直接影响 KV cache、排队和异构 TP 的收益，因此只使用平均长度会降低规划精度。

3.2 训练和 Rollout 配置

训练配置由 TP、PP、CP、DP、micro-batch、ZeRO level 表示，训练设备数为 $TP \times PP \times CP \times DP$ 。当前 Megatron-Core NPU 引擎在成本模型侧过滤 $PP \neq 1$ 的候选，因为该引擎当前只支持 $PP=1$ 。

Rollout 用 TP 列表表示实例，例如 $[4, 2, 2]$ 表示一个 TP4 和两个 TP2 实例，占用 8 个设备。require_heterogeneous_rollout_tp 可以要求列表中包含不同 TP 值。

3.3 计划结果

计划包含训练配置、Rollout TP 列表、训练时间、Rollout 时间、全局时间、设备数、最大并发和重配置收益。核心目标为：

```
T_global = max(T_train, T_rollout)
```

只有当收益超过 min_gain_ratio 并足以覆盖 reconfiguration_cost_s 时，规划器才建议切换当前计划。

4 内置解析成本模型

4.1 Rollout 模型

RolloutCostModel 将 Rollout 拆成 prefill 和 decode。KV cache 容量估计为：

```
KV pool = TP × 单设备容量 - 模型权重 - 激活工作区  
token capacity = KV pool / 每 token KV 字节数 × (1 - 碎片率)
```

prefill 估计使用层数、隐藏维度、prompt token、分块长度、峰值算力和利用率；decode

估计使用权重读取量、活跃 token、内存带宽和 TP

通信。请求按总长度降序排列，再按实例吞吐贪心分配，最终取各实例 makespan 最大值。

该模型能够表达不同 TP 实例的容量、吞吐和长请求分配，但不等价于真实 vLLM scheduler、CANN kernel 或 HCCL 网络仿真。

4.2 训练模型和内存预检

TrainingCostModel 使用模型参数量、层数、隐藏维度、设备算力、带宽、TP/PP/DP 和 profiling 系数估计迭代时间。内存模型覆盖权重、梯度、优化器状态、分片、activation、workspace、碎片率、Megatron-Core 构建瞬态和 recompute_logprobs 的 logits 临时张量。

候选训练配置若触发 OOM 会被剪枝。NPU 分支还支持 memory budget preflight，通过一次完整 recompute probe 检查启动时峰值，弥补静态估算对 allocator 峰值的低估。

4.3 NPU 硬件配置注意事项

分支提供 hardware_config/ascend_910b3_64g.yaml，包含 Ascend 910B3

级别容量、算力、带宽和每节点 8 卡配置。需要特别注意：部分早期文件名含 npu 的验证配置仍引用 hardware_config/a100_80g.yaml，并使用 FSDP、磁盘同步等历史路径。这些配置不能直接作为 Ascend 性能基准。

正式实验必须保证硬件 YAML、device_backend: npu、distributed_backend: hccl、训练后端和 profiling 参数与真实设备一致，并用实测数据校准成本模型。

5 Sailor 和 Vidur 适配器

5.1 统一接口

HybridSimulatorCostModel 对上层提供：

```
evaluate_training(train_config, B_global, L)  
evaluate_rollout(rollout_config, requests)
```

后端	作用	依赖
analytic	仓库内解析估算	无外部仿真项目
sailor	训练迭代时间估算	Sailor 路径、Python 和命令
vidur	Rollout makespan 估算	Vidur 路径、Python 和命令

外部调用失败时，simulator_allow_fallback: true 会记录 requested_backend 和 fallback_reason，再回退到解析模型；设为 false 可用于严格实验。

5.2 Sailor 契约

适配器为每个训练候选生成 input.json，包含训练并行、batch size、平均序列长度、硬件、模型和 profiling。命令模板可使用 {input_json}、{output_json}、{sailor_path}、{rl_framework_path} 和 {python}。输出可写 output.json，或通过 SAILOR_RESULT_JSON= 输出。接受 t_train、iteration_time、iteration_time_s、iteration_time_sec 或 throughput。

5.3 Vidur 契约

适配器把请求写成字段为 arrived_at、num_prefill_tokens、num_decode_tokens 的 trace CSV。命令可使用 trace、output JSON、output directory、TP 列表、实例数和最大 TP。输出读取顺序为 output.json、VIDUR_RESULT_JSON=、request_metrics.csv、batch_metrics.csv，统一转换为 makespan 秒数。

5.4 边界

命令契约只规定输入输出，不自动保证外部工具已经支持 Ascend 设备、HCCL、vLLM-Ascend 或真实 NPU 内存行为。分支中的部分 Vidur 示例默认值仍是 A100/Llama 风格，接入 NPU 实验时必须替换设备、模型和 profile。

6 GRP 搜索和动态重规划

6.1 两层搜索

TwoLevelNestedOptimizer 先枚举训练 TP/PP/DP/micro-batch，执行 OOM 和能力剪枝，再用剩余设备生成 Rollout TP 分区。每个候选都估计 T_train 和 T_rollout，以两者最大值排序。搜索结果保留探索数、OOM 剪枝数、提前停止数和优化耗时。

6.2 在线触发

信号	作用
queue pressure	待处理队列相对容量的压力
active rollout pressure	运行中请求相对并发上限的压力
rejected rollout delta	新增拒绝请求数
rollout/train imbalance	两阶段耗时失衡
interval	到达固定重规划间隔

warmup、最少历史、cooldown、最小收益比例和重配置成本共同限制频繁切换。PreflightPlanner 可在启动前使用历史或合成请求生成配置；设备物化脚本再将 TP 列表装箱到具体主机、设备、rank 和端口。

7 NPU 运行链路和仿真关系

engine/device_utils.py 统一设备识别、绑定、同步、设备计数和内存统计；NPU 默认通信后端为 HCCL。megatron_npu_compat.py 对部分第三方 CUDA 状态、设备移动、随机数、stream、event 和显式 CUDA 字面量做 NPU 映射。该方案减少第三方源码改动，但对 Megatron-Core、Megatron Bridge 和 torch-npu 版本敏感。

训练侧以 Megatron-Core、Qwen3/Qwen3-MoE、TP/EP/DP、distributed optimizer、CPU optimizer offload 和分布式 checkpoint 为主。Rollout 侧通过 vLLM-Ascend 提供 OpenAI-compatible 服务。

HCCL 原位同步中 trainer 是 rank 0，vLLM TP workers 使用连续 rank；流程包括 pause、start update、流式发送、finish 和 resume。communicator 会复用拓扑签名，因此 GRP 改变 Rollout worker 或 TP 拓扑后，必须在下一次传输前重启 HCCL Rollout group。仿真的重配置成本应包含这个重启和模型加载代价。

8 安装和运行

8.1 CPU 解析仿真

```
git clone https://github.com/NetX-lab/Libra.git
cd Libra
git checkout NPU_Support
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements.txt
export PYTHONPATH="$(dirname "$PWD"):${PYTHONPATH:-}"
python examples/global_resource_planner_full_flow.py
```

8.2 Ascend 环境

```
export ASCEND_SET_ENV=/path/to/ascend-toolkit/set_env.sh
bash scripts/setup_npu_env.sh
source .venv-npu/bin/activate
```

官方 HCCL profile 使用独立环境：

```
export ASCEND_SET_ENV=/path/to/cann-9.0/set_env.sh
NPU_STACK_PROFILE=hccl INSTALL_MEGATRON_NPU=1 bash scripts/setup_npu_env.sh
```

legacy profile 和 HCCL profile 的 Python、PyTorch、torch-npu、vLLM 和 vLLM-Ascend 版本不同，不应混装。

8.3 启动前规划

```
PYTHONPATH="$(dirname "$PWD")" python scripts/run_global_resource_preflight.py \
--config configs/r2e_gym_qwen3_14b_mcore_npu_5node40_smoke.yaml \
--output-config runs/preflight/planned.yaml \
--decision-json runs/preflight/decision.json \
--synthetic-requests 32 --synthetic-input-len 1024 --synthetic-output-len 2048
```

真实 history JSONL 应包含 input_len 和 output_len。生产 NPU 入口包括 run_npu_ablation.sh、run_multinode_r2e_mcore_npu.sh、run_vllm_ascend_server.sh 和 HCCL/preflight 校验脚本。

9 验证结果

验证项	结果	说明
compileall	PASS	分支源码字节码编译通过
CPU 友好测试	53 passed, 10 skipped	规划器、适配器、调度器和弹性执行器相关测试
GRP 完整流程	FLOW_OK	Sailor/Vidur 命令契约、候选搜索和计划回写
真实 NPU 多节点	未执行	当前主机没有 Ascend/CANN/HCCL 环境

自带完整流程使用 4 条长请求和 4 个逻辑设备，示例输出为：训练 TP1 PP1 DP2 micro 1，占 2 个设备；Rollout [1, 1]，占 2 个设备；T_train=104.1440s、T_rollout=249.2901s、T_global=249.2901s；决策为 same_plan，最后输出 FLOW_OK。这些秒数来自确定性命令契约模型，仅用于软件流程回归。

需要 PyTorch 的测试未作为本次 CPU 通过项统计，因为执行环境没有安装 torch。HCCL all-reduce、vLLM-Ascend 服务、Megatron-Core NPU 初始化和多节点训练需要在目标环境单独验收。

10 输出和可观测性

主要输出包括 history/*.jsonl、global_resource_rollout_manifest.json、runtime_reconfiguration_events.jsonl、rollout_process_registry.jsonl、cmlfq_tree/*.json、phase_trace_rank*.jsonl、启动前 decision.json 和设备 placement.json。这些数据用于验证成本模型、比较不同资源配置、分析长度分布和复盘重配置。

建议记录训练/Rollout 时间、token/s、trajectory/s、各 TP 请求数、HCCL transfer 时间、OOM 预检与实际 OOM、重配置耗时、队列压力、拒绝数、policy version lag 和长尾请求比例，并至少计算 P50/P95/P99。

11 适用性与限制

适合用途包括启动前资源比例搜索、TP/DP/micro-batch 筛选、异构 Rollout 组合比较、合成或真实 history 规划、外部适配器回归、OOM 预警和 A/B 配置生成。

它不能单独证明 CANN kernel、torch-npu 算子、HCCL 多机稳定性、vLLM-Ascend 调度、原位权重刷新一致性、cluster-swap 异常恢复或最终训练质量。

主要误差来自硬件 profile 未校准、贪心 Rollout 分配近似真实队列、长尾请求分布变化、HCCL 拓扑重建成本低估，以及部分配置沿用 A100/Llama 示例。若 reconfiguration_cost_s 太小，规划器可能频繁选择实际代价很高的拓扑切换。

12 改进和验收建议

立即改进：

- 对 device_backend: npu 与 A100 hardware profile 的组合增加配置警告或拒绝；
- 在启动日志中输出 CANN、Python、PyTorch、torch-npu、MCore、vLLM 和 vLLM-Ascend 版本；
- 记录 HCCL communicator 重建、vLLM reload/restart、health check、drain 和模型加载时间；
- 按模型、序列长度、TP 拓扑保存 NPU profile，并用 history 自动校准；
- 为共享控制文件加入 attempt id、lease 和过期清理。

建议按单卡环境检查、MCore 初始化、vLLM-Ascend smoke、HCCL 通信、端到端小步数、固定布局与 GRP A/B、C-MLFQ A/B、EHP/cluster-swap、目标规模运行、真实 history 再校准的顺序验收。

13 总结

NPU_Support 的仿真工具是 Libra 内嵌的资源规划和成本估计子系统，不是图形化或 cycle-level NPU 模拟器。它以请求长度和硬件/模型配置为输入，通过解析模型或 Sailor/Vidur 契约估计训练与 Rollout，再由两层嵌套优化器搜索计划，并由 GRP 根据历史、压力、收益和切换成本决定是否重配置。

当前软件逻辑已具备可复现验证基础。下一步重点应放在 Ascend 真实 profile、HCCL 权重同步和拓扑重建成本、长尾请求级排队模型，以及固定布局/GRP/C-MLFQ/EHP 的同条件 A/B 实测。真实

NPU 校准完成前，报告中的时间数字只能作为流程验证和相对排序依据。

14 参考资料

1. Libra NPU_Support 分支
2. 固定代码提交
3. 成本模型
4. GRP
5. 仿真器适配器
6. 优化器
7. NPU 环境说明
8. 集群运行手册